ChatGPT Share

Optimization Pass Methodology.pdf
PDF

main.pdf
PDF

so now I want to include a reinforcement learning prototype model in order to show the utility of QLego for phase reordering problem. So let me define exact markovian process

so now I want to include a reinforcement learning prototype model in order to show the utility of QLego for phase reordering problem. So let me define exact markovian process
Init -> Translation -> Layout (Self Loop) -> Routing(Self Loop) -> Optimization states(each self loop) -> Scheduling -> Terminate.....

so we want to come up with optimal set of Layout -> Routing -> Optimization sequences..... formulate the RL problem and suggest how should I approach this?

I'm going to ground this in your uploaded material first, then turn it into a concrete RL formulation with state, action, reward, and rollout structure that fits QLego.

Your uploaded draft already gives the right motivation: in quantum compilation, pass effects are not independent, the best ordering is circuit- and hardware-specific, and QLego is useful precisely because it can execute cross-framework sequences as an RL environment. It also already positions MQT Predictor and newer RL work as close prior art for pass-sequence learning. main main

The main design choice I'm making here is: don't model this as a totally free pass-by-pass MDP at first. For your paper, a **stage-constrained MDP** is cleaner, smaller, and matches your proposed skeleton much better.

Thought for 14s

Yes — this is a very natural RL problem for QLego.

Given your pipeline

$$\text{Init} \to \text{Translation} \to \text{Layout}^* \to \text{Routing}^* \to \text{Optimization}^* \to \text{Scheduling} \to \text{Terminate}$$

the best formulation is not a completely unrestricted MDP over all passes, but a **stage-constrained episodic MDP**. That aligns with your own methodology notes: use a fixed skeleton, search over stage choices, and allow self-loops only inside stages where repeated improvement makes sense.
Optimization Pass Methodology Optimization Pass Methodology

1. What is the RL problem?

The agent should learn:

$$\pi(a_t \mid s_t)$$

where each action chooses the next compiler operation under stage constraints, and the episode reward reflects final compiled-circuit quality.

The goal is:

$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{T} \gamma^t r_t\right]$$

with an episode ending when the agent emits `advance_stage` through Scheduling and then `terminate`.

---

2. MDP components

State

A good state should contain:

1. **Current stage**

$$g_t \in \{\text{translation}, \text{layout}, \text{routing}, \text{optimization}, \text{scheduling}, \text{done}\}$$

2. **Circuit features** after the current partial pipeline, for example:
   - number of qubits
   - number of gates
   - number of 2-qubit gates
   - depth
   - critical depth
   - number of SWAPs
   - native-basis fraction
   - measurement count
   - reset count
   - Clifford fraction
   - parameterized-gate fraction
   - connectivity violation count before routing
   - estimated duration

- estimated fidelity / error proxy

3. **Hardware features**
   - number of physical qubits
   - coupling density
   - average 2Q error
   - average readout error
   - gate durations
   - basis gate set
4. **Pipeline history**
   - which passes/bundles already used
   - repetition count of current stage
   - total step count
   - whether layout has already been committed
   - whether routing has already introduced swaps
   - whether basis translation has already happened

So the state can be written as

$$s_t = \big(g_t,\ \phi(C_t),\ \psi(H),\ h_t\big)$$

where C_t is the current circuit, H the backend, $\phi(\cdot)$ circuit features, and h_t pass-history metadata.

This is also consistent with the feature-driven view already mentioned in your draft and methodology notes. main Optimization Pass Methodology

---

Action space

At each stage, only a subset of actions is legal.

Translation stage
Actions:

- choose translator / unroller
- `advance_stage`

Example:

$$\mathcal{A}_{\text{translation}} = \{\text{qiskit basis translate}, \text{bqskit rebase}, \text{none}, \text{advance}\}$$

Layout stage
Actions:

- choose a layout pass from any SDK
- repeat another layout pass
- `advance_stage`

$$\mathcal{A}_{\text{layout}} = \{\text{SabreLayout}, \text{DenseLayout}, \text{VF2Layout}, \text{TKET placement}, \text{BQSKit placement}, \text{advance}\}$$

Routing stage
Actions:

- choose a router
- route again if legal
- `advance_stage`

Optimization stage
This is where RL matters most.

Do **not** use raw individual passes first. Use **bundle options** as actions:

- `1q_simplify`
- `commutation_cleanup`
- `clifford_cleanup`
- `2q_block_collect`
- `target_rebase`
- `measurement_cleanup`
- `final_cleanup`
- `advance_stage`

This follows your earlier recommendation that optimization should be modeled as staged bundles rather than raw subsets. Optimization Pass Methodology Optimization Pass Methodology

So formally, the action set is stage-dependent:

$$a_t \in \mathcal{A}(g_t)$$

and illegal actions are masked.

Transition

A transition is deterministic given the current circuit and chosen pass, aside from possible nondeterminism in heuristic compilers.

$$s_{t+1} = P(s_t, a_t)$$

where P means:

1. run the selected QLego pass/bundle,
2. update the circuit,
3. extract new features,
4. either stay in stage or move to next stage.

If a pass fails, you can:

- assign a penalty and terminate, or
- assign penalty and keep state unchanged.

For prototype work, “penalty and keep state unchanged” is easier.

Reward

Use a **dense incremental reward** plus a **final terminal reward**.

A good incremental reward is improvement in a weighted cost:

$$r_t = F(C_t) - F(C_{t+1}) - \lambda$$

where λ is a small step penalty, and

$$F(C) = w_1 \cdot 2\mathrm{Q}(C) + w_2 \cdot \mathrm{Depth}(C) + w_3 \cdot \mathrm{Duration}(C) + w_4 \cdot \mathrm{ErrorProxy}(C)$$

This is directly aligned with your earlier multi-objective scoring idea. Optimization Pass Methodology

A more explicit normalized version:

$$r_t = w_1 \frac{\Delta 2\mathrm{Q}}{2\mathrm{Q}_0} + w_2 \frac{\Delta \mathrm{Depth}}{\mathrm{Depth}_0} + w_3 \frac{\Delta \mathrm{Duration}}{\mathrm{Duration}_0} + w_4 \frac{\Delta \mathrm{FidelityProxy}}{|\mathrm{FP}_0|} - \lambda$$

where $\Delta x = x_t - x_{t+1}$.

Then add terminal reward:

$$r_T = \beta \cdot \mathrm{Score}(C_T)$$

with bonus for legal completion.

---

3. The cleanest formulation for your paper

I would present it as a **finite-horizon stage-aware MDP with action masking**.

Why this is better than a free-form MDP

Because your compilation flow is not arbitrary:

- layout should happen before routing
- routing before final scheduling
- optimization can loop
- some passes are only meaningful after certain structural changes

Your own notes already argue that stage structure is the right abstraction and that optimization should not be treated as flat alternative choice. Optimization Pass Methodology

So define the episode as:

$$\tau = (s_0, a_0, r_0, s_1, \ldots, s_T)$$

with hard stage order:

$$\mathrm{translation} \prec \mathrm{layout} \prec \mathrm{routing} \prec \mathrm{optimization} \prec \mathrm{scheduling}$$

and self-loops allowed only within selected stages.

---

4. Recommended prototype environment for QLego

Environment interface

Make QLego expose something like:

```python
obs = env.reset(circuit, backend)

obs, reward, done, info = env.step(action)
```

Where `obs` includes:

- current stage id
- feature vector
- legal action mask
- partial pipeline history

Action granularity

For the first paper:

- layout: individual pass choices are okay
- routing: individual pass choices are okay
- optimization: use **bundle actions**, not raw passes

That gives you a manageable search space and a cleaner story.

---

5. Suggested state feature vector

I would use something like this:

$$\phi(C_t, H) = [n_q,\ n_g,\ n_{1q},\ n_{2q},\ \mathrm{depth},\ \mathrm{critdepth},\ n_{\mathrm{swap}},\ \mathrm{clifford_ratio},\ \mathrm{param_ratio},\ \mathrm{measure_count},\ \mathrm{reset_count},\ \mathrm{native_fraction},\ \mathrm{violati}$$

plus history flags such as:

- used_sabre_layout

- used_basic_swap
- used_commutation_cleanup
- optimization_loop_count
- total_steps

This is enough for a prototype PPO/DQN agent.

---

6. Which RL algorithm?

For your prototype:

Best first choice: PPO

Why:

- discrete actions
- variable episode lengths
- easy action masking
- robust in practice

Simpler baseline: DQN

Use only if action space is small and fixed.

Since your action set changes by stage, PPO with masking is cleaner.

So I'd recommend:

- **RL method:** PPO
- **baseline 1:** random legal policy
- **baseline 2:** greedy one-step improvement
- **baseline 3:** fixed SDK presets
- **baseline 4:** exhaustive or beam search on small search spaces

Your draft already positions QLego well for this kind of comparison because it can execute cross-framework sequences while existing toolchains typically cannot. main main

---

7. What exactly should the agent optimize?

You have three possible objectives.

Option A: depth-centric

$$\mathrm{Score}(C) = -\mathrm{Depth}(C)$$

Too narrow.

Option B: weighted NISQ score

$$\mathrm{Score}(C) = -w_1 2\mathrm{Q} - w_2 \mathrm{Depth} - w_3 \widehat{\mathrm{Duration}} - w_4 \widehat{\mathrm{Error}}$$

Best for prototype.

Option C: Pareto / multi-objective RL

More ambitious, not needed initially.

Use Option B first.

---

8. Important modeling choices

Choice 1: include "advance stage" as an action

Yes. Very important.

Otherwise the agent cannot learn when to stop looping in layout/routing/optimization.

Choice 2: allow repeated same pass?

Yes, but cap it.

Example:

- max 2 layout actions
- max 2 routing actions
- max 5 optimization actions

Otherwise the agent may cycle forever.

Choice 3: allow going back from routing to layout?

For prototype: **no**.

Keep the stage order acyclic except self-loops.
This makes the environment much easier.

Choice 4: use bundles or raw passes?

- layout/routing: raw passes okay
- optimization: bundles first

That is the cleanest compromise.

---

9. A strong paper framing

You can say something like:

> We formulate cross-framework compiler phase ordering in QLego as a stage-constrained Markov Decision Process. The agent observes circuit, hardware, and compilation-history features, and selects legal pass actions within a fixed compilation skeleton. Self-loops are permitted in layout, routing, and optimization stages to model iterative improvement, while action masking enforces semantic stage constraints. Rewards are defined using incremental improvement in a multi-objective NISQ cost combining two-qubit count, depth, estimated duration, and fidelity proxy.

That is solid and directly connected to your phase-ordering motivation. Your draft already states that fixed compiler orders are suboptimal and that QLego provides infrastructure both for exhaustive search and ML-guided ordering. main

---

10. Minimal prototype I would build

I would not start with the full pipeline.

Prototype v1

Only learn:

$$\text{Layout} \to \text{Routing} \to \text{Optimization}^*$$

with:

- 3–5 layout choices
- 3–5 routing choices
- 5–8 optimization bundles
- max episode length 8 or 10

Why

Because this is enough to demonstrate the core utility of QLego:
cross-framework phase ordering over modular passes.

Your results section already shows that no single pass dominates and that cross-SDK combinations can beat same-SDK pipelines depending on the circuit. That is exactly the experimental phenomenon RL can exploit. main

---

11. Concrete mathematical formulation

You can define the MDP as:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \rho_0, \gamma)$$

where

States

$$s_t = (g_t, \phi(C_t), \psi(H), h_t)$$

Actions

$$a_t \in \mathcal{A}(g_t)$$

Transition

$$C_{t+1} = \text{Apply}(C_t, a_t)$$

$$s_{t+1} = (g_{t+1}, \phi(C_{t+1}), \psi(H), h_{t+1})$$

Reward

$$r_t = \text{Score}(C_{t+1}) - \text{Score}(C_t) - \lambda$$

with

$$\text{Score}(C) = -\alpha_1\, 2\text{Q}(C) - \alpha_2\, \text{Depth}(C) - \alpha_3\, \widehat{\text{Duration}}(C) - \alpha_4\, \widehat{\text{Error}}(C)$$

Objective

$$\pi^* = \arg\max_{\pi} \mathbb{E}_\pi \left[\sum_{t=0}^{T} \gamma^t r_t \right]$$

---

12. My recommendation for your exact setting

For QLego, I would do this:

Environment

- fixed stage order
- masked legal actions
- self-loops in routing and optimization
- optional self-loop in layout, capped at 2

Actions

- individual layout passes
- individual routing passes
- optimization bundles

Reward

- dense shaped reward based on normalized improvements

- small step penalty
- terminal bonus for successful completion

Agent

- PPO

Baselines

- Qiskit preset pipelines
- TKET default pipelines
- BQSKit defaults
- greedy search
- beam search

Evaluation

- stratify by circuit family and qubit count
- report depth, 2Q count, duration proxy, fidelity proxy
- show learned policy sometimes prefers same-SDK and sometimes cross-SDK pipelines

That will strongly demonstrate the utility of QLego.

If you want, I can next write this as a clean paper subsection with notation and equations in LaTeX.

Sources